DATA STRUCTURES

Assignment 6

Mahmoud Ahmed
202500283

Creating a Struct

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct Task{
6      int pid;
7      string description;
8      Task* next;
9  };
10
11 Task* top = nullptr;
12
```

Adding Functions

```
void addTask(Task t){ // push
    Task* new_task = new Task(t);
    new_task->pid = t.pid;
    new_task->description = t.description;
    new_task->next = top;
    top = new_task;
}

void processTask(){ // pop
    if (top == nullptr){
        cout << "Stack UNDERFLOW!\n";
        return;
    }
    Task* temp = top;
    cout << "Task was processed:\nPID: " << temp->pid << "\nDescription: " << temp->description;
    cout << "1";
    top = top->next;
    delete temp;
}
```

Adding Functions

```
void displayTasks(){
    if (top == nullptr){
        cout << "Stack is Empty\n";
        return;
    }
    Task* temp = top;
    for (int i = 0; temp != nullptr; i++){
        cout << "--- Task " << i + 1 << "---\nPID: " << temp->pid << "\nDescription: " << temp->description;
        cout << '\n';
        temp = temp->next;
    }
}

void processAll(){
    if (top == nullptr){
        cout << "Stack Already Empty\n";
        return;
    }
    while (top != nullptr){
        processTask();
    }
}
```

Main Function

```
int main(){
    cout << "========== Simple Task Manager ==========\n";
    bool card = true;
    while (card){
        cout << "Choose an option:\n";
        cout << "1- Add a task\n";
        cout << "2- Process a task\n";
        cout << "3- Display tasks\n";
        cout << "4- Process all tasks\n";
        cout << "5- Exit\n";
        int option;
        cin >> option;
        cout << '\n';
```

Main Function

```
        switch(option){
            case 1: {
                cout << "Task PID: ";
                int id;
                cin >> id;
                cout << "Task Description: ";
                string description;
                cin.ignore(1, '\n');
                getline(cin, description);
                Task t = {id, description};
                addTask(t);
                break;
            }
            case 2: {
                processTask();
                break;
            }
            case 3: {
                displayTasks();
                break;
            }
            case 4: {
                processAll();
                break;
            }
            case 5: {
                card = false;
                break;
            }
            default: {
                cout << "Invalid option\n";
```

Main Function

```
            }
            default: {
                cout << "Invalid option\n";
            }
        }
        cout << '\n';
    }
    return 0;
}
```

THANK YOU

MAHMOUD AHMED
202500283